Correção do exercício 9

Três conferências para cada procedure

1º passo

As regras antes da ação

Os IF com RAISE EXCEPTION vêm antes do INSERT e do UPDATE?

2º passo

Conferir com SELECT

Depois de cada CALL, um SELECT mostra o que mudou?

3º passo

O teste que falha

A mensagem diz o motivo com as palavras da loja, e nada foi gravado?

Triggers

Banco de Dados 2 · Aula 20

SQL: Structured Query Language · PL/pgSQL: Procedural Language/PostgreSQL

NOVO ASSUNTO

Trigger

Na procedure, alguém precisa lembrar de chamar o CALL. Mas algumas regras têm de valer sempre, venha a mudança de onde vier:

- O banco roda a função sozinho, a cada INSERT ou UPDATE na tabela.
- Ninguém chama o trigger (gatilho): basta o dado mudar.

Primeiro, um trigger que registra o que mudou: a auditoria do status do pedido.

Onde e por que usar um trigger

Três usos na Loja Virtual

Auditoria

Registrar o que mudou

Guarda o status antes e depois, quem mudou e quando.

Regra

Recusar o dado errado

Só quem comprou o produto pode avaliar.

Padrão

Corrigir o dado

O código do cupom fica sempre em maiúsculas.

Quem mudou o status deste pedido?

A pergunta que as tabelas de hoje não respondem

Hoje

As tabelas não guardam

O pedido mostra o status de agora, mas não o de antes, nem quando mudou.

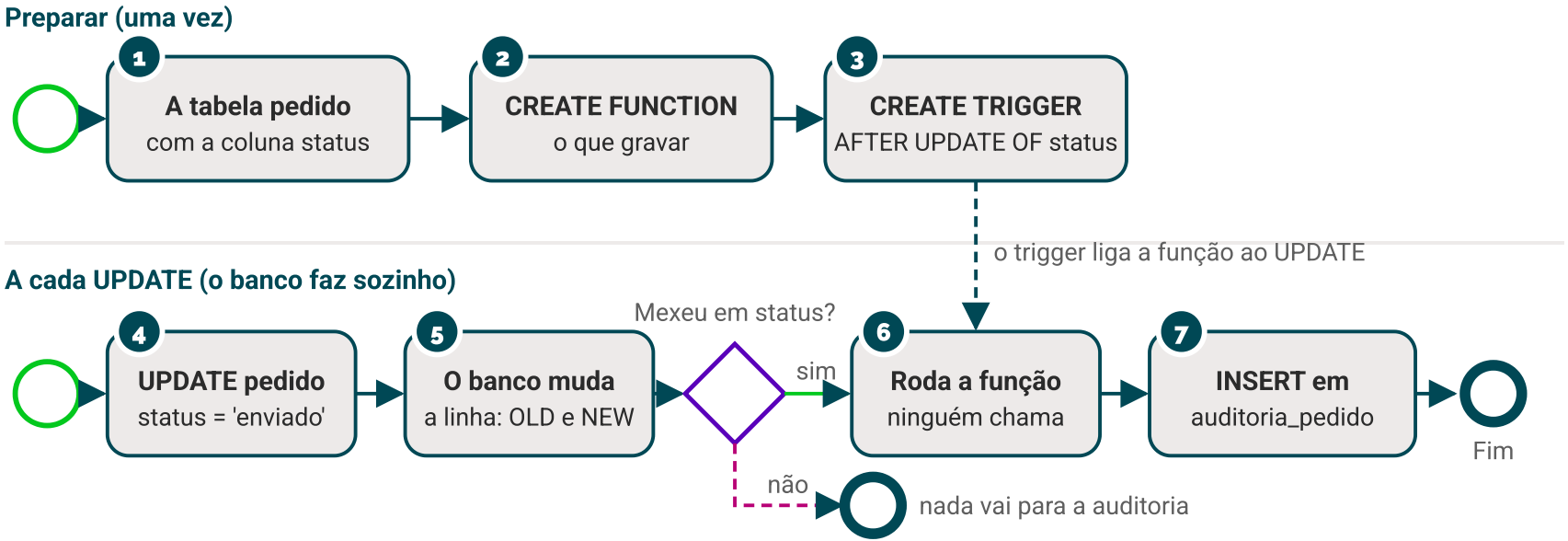
Com o trigger

Cada mudança vira uma linha

A tabela auditoria_pedido, que já existe na base, guarda o antes, o depois, quem e quando.

O caminho de um trigger

Primeiro se prepara uma vez; depois o banco age sozinho a cada UPDATE



> Os passos 1 a 3 rodam uma vez. Do 4 em diante, ninguém escreve CALL: quem dispara a função é o próprio UPDATE.

1ª parte: a função do trigger (1 de 2)

OBJETIVO **O que fazer quando o status mudar?**

```
CREATE OR REPLACE FUNCTION   -- cria ou troca
  fn_auditar_status_pedido()   -- sem parâmetros
RETURNS trigger   -- o tipo especial
LANGUAGE plpgsql AS $$ BEGIN   -- corpo
```

A função do trigger não recebe parâmetro e devolve o tipo especial trigger.

1ª parte: a função do trigger (2 de 2)

OBJETIVO **Gravar o antes e o depois**

```
1 INSERT INTO auditoria_pedido   -- grava
2   (pedido_id, status_antes,    -- colunas
3   status_depois)   -- e o depois
4 VALUES (NEW.id, OLD.status,    -- o antes
5   NEW.status);   -- o depois
6 RETURN NULL; END; $$;   -- AFTER: NULL
```

OLD é a linha antes da mudança; NEW é a linha depois. No AFTER, o retorno é ignorado: RETURN NULL.

2ª parte: o CREATE TRIGGER

OBJETIVO **Ligar a função à tabela e ao evento**

```
1 DROP TRIGGER IF EXISTS   -- apaga se existe
2   trg_auditar_status_pedido ON pedido;   -- nome
3 CREATE TRIGGER trg_auditar_status_pedido   -- cria
4 AFTER UPDATE OF status ON pedido   -- quando
5 FOR EACH ROW EXECUTE FUNCTION   -- a cada linha
6   fn_auditar_status_pedido();   -- roda a função
```

AFTER UPDATE OF status: depois de um UPDATE que mexe na coluna status. O DROP TRIGGER IF EXISTS deixa o arquivo rodar de novo.

Ninguém chama o trigger

OBJETIVO **Um UPDATE comum basta para a auditoria aparecer?**

```
1 UPDATE pedido SET status = 'enviado',   -- muda
2   enviado_em = LOCALTIMESTAMP   -- e a data
3 WHERE id = 2;   -- do pedido 2
4 SELECT pedido_id, status_antes,   -- confere
5   status_depois, usuario   -- o que foi gravado
6 FROM auditoria_pedido;   -- 2, pago, enviado
```

Preveja a linha antes de rodar. Resultado: 2, pago, enviado e o usuário que fez o UPDATE.

O que aconteceu no pedido 2

OBJETIVO **O caminho do trigger com os valores reais**

Antes
pedido 2
status = 'pago'

→

O UPDATE
SET status = 'enviado'
WHERE id = 2

→

Dentro da função
OLD.status = 'pago'
NEW.status = 'enviado'
NEW.id = 2

Depois
pedido 2
status = 'enviado'

Nova linha em auditoria_pedido
pedido_id 2 · antes pago · depois enviado
usuario postgres · quando: a data e a hora do UPDATE

A função só passou três valores. usuario e quando vêm do DEFAULT da tabela: current_user (quem está conectado) e now() (a data e a hora de agora).

FOR EACH ROW e UPDATE OF

OBJETIVO **Quantas linhas de auditoria cada UPDATE grava?**

```
1 UPDATE pedido SET status = 'entregue',   -- dois
2   entregue_em = LOCALTIMESTAMP   -- e a data
3 WHERE id IN (5, 6);   -- de uma vez
4 UPDATE pedido SET dest_numero = '100'   -- outra
5 WHERE id = 3;   -- não é o status
6 SELECT COUNT(*) FROM auditoria_pedido;   -- 3
```

O primeiro UPDATE grava 2 linhas, uma por pedido (FOR EACH ROW). O segundo não grava nada: não mexeu no status.

NOVO ASSUNTO

Trigger BEFORE

O AFTER roda depois que o dado foi gravado: ele registra. Para impedir um dado errado, o trigger tem de rodar antes:

- Com RAISE EXCEPTION, ele recusa: a linha não entra.
- Com NEW.coluna := ..., ele corrige o valor antes de gravar.

No BEFORE, o trigger termina com RETURN NEW: é a linha que vai ser gravada.

Recusar: só quem comprou avalia (1 de 3)

OBJETIVO **Este cliente comprou o produto que quer avaliar?**

```
1 CREATE OR REPLACE FUNCTION   -- cria ou troca
2   fn_avaliacao_so_quem_comprou()   -- sem parâmetros
3 RETURNS trigger   -- tipo especial
4 LANGUAGE plpgsql AS $$ BEGIN   -- corpo
```

Os dados chegam em NEW: a avaliação que vai ser gravada.

Recusar: só quem comprou avalia (2 de 3)

OBJETIVO **Existe uma venda deste produto para este cliente?**

```
1 IF NOT EXISTS (SELECT 1   -- comprou?
2   FROM item_pedido i   -- os itens
3   JOIN pedido p ON p.id = i.pedido_id   -- pedidos
4   WHERE p.cliente_id = NEW.cliente_id   -- cliente
5     AND i.produto_id = NEW.produto_id   -- produto
6     AND p.status IN ('pago',   -- venda
7       'enviado', 'entregue')) THEN   -- fecha
```

Leia em voz alta: «se não existe item deste produto num pedido pago, enviado ou entregue deste cliente...».

Recusar: só quem comprou avalia (3 de 3)

OBJETIVO **Recusar ou deixar a linha entrar**

```
    RAISE EXCEPTION   -- recusa
    'Cliente % não comprou o produto %',   -- texto
      NEW.cliente_id, NEW.produto_id;   -- os %
  END IF;   -- fim do teste
  RETURN NEW;   -- a linha entra
END; $$;   -- fim
```

No BEFORE, o RAISE EXCEPTION impede o INSERT. Se passou no teste, RETURN NEW deixa a linha entrar.

O trigger BEFORE INSERT

OBJETIVO **Rodar a função antes de gravar cada avaliação**

```
CREATE TRIGGER   -- cria
  trg_avaliacao_so_quem_comprou   -- o nome
BEFORE INSERT ON avaliacao   -- antes de gravar
FOR EACH ROW EXECUTE FUNCTION   -- a cada linha
  fn_avaliacao_so_quem_comprou();   -- a função
```

BEFORE INSERT: antes de gravar, dá tempo de recusar. Antes dele, o DROP TRIGGER IF EXISTS de sempre.

Um teste que passa e um recusado

OBJETIVO **Qual avaliação entra?**

```sql
INSERT INTO avaliacao (cliente_id,   -- avaliação
  produto_id, nota) VALUES (2, 19, 5);   -- comprou
INSERT INTO avaliacao (cliente_id,   -- outra
  produto_id, nota) VALUES (2, 1, 1);   -- recusa
SELECT COUNT(*) FROM avaliacao;   -- 51 (eram 50)
```

O segundo INSERT para com: Cliente 2 não comprou o produto 1. Só a primeira avaliação entrou.

Corrigir: o código do cupom em maiúsculas

OBJETIVO **Guardar o código sempre igual, digitado de qualquer jeito**

```
1 CREATE OR REPLACE FUNCTION   -- cria ou troca
2   fn_cupom_codigo_padrao()   -- sem parâmetros
3 RETURNS trigger LANGUAGE plpgsql AS $$   -- trigger
4 BEGIN   -- começa
5   NEW.codigo :=   -- troca o valor
6     UPPER(TRIM(NEW.codigo));   -- por este
7   RETURN NEW;   -- a linha corrigida
8 END; $$;   -- fim
```

UPPER e TRIM são da aula de funções SQL (*Structured Query Language*).

BEFORE INSERT OR UPDATE

OBJETIVO **Vale no cadastro e na alteração?**

```
1 DROP TRIGGER IF EXISTS   -- apaga se existe
2   trg_cupom_codigo_padrao ON cupom;   -- nome
3 CREATE TRIGGER trg_cupom_codigo_padrao   -- cria
4 BEFORE INSERT OR UPDATE ON cupom   -- os dois
5 FOR EACH ROW EXECUTE FUNCTION   -- a cada linha
6   fn_cupom_codigo_padrao();   -- a função
```

INSERT OR UPDATE: o trigger corrige o código no cadastro e em qualquer alteração.

O trigger corrige antes de gravar

OBJETIVO **Um código digitado errado continua certo no banco?**

```sql
UPDATE cupom   -- muda o cupom
SET codigo = '  vip25 '   -- digitado errado
WHERE codigo = 'VIP25';   -- o VIP25
SELECT codigo FROM cupom   -- confere
WHERE codigo = 'VIP25';   -- continua VIP25
```

O trigger trocou vip25 por VIP25 antes de gravar.

AFTER ou BEFORE?

Sempre duas partes: a função (RETURNS trigger) e o CREATE TRIGGER

AFTER

Registra

Roda depois de gravar: histórico ou mudança em outra tabela. Termina com RETURN NULL.

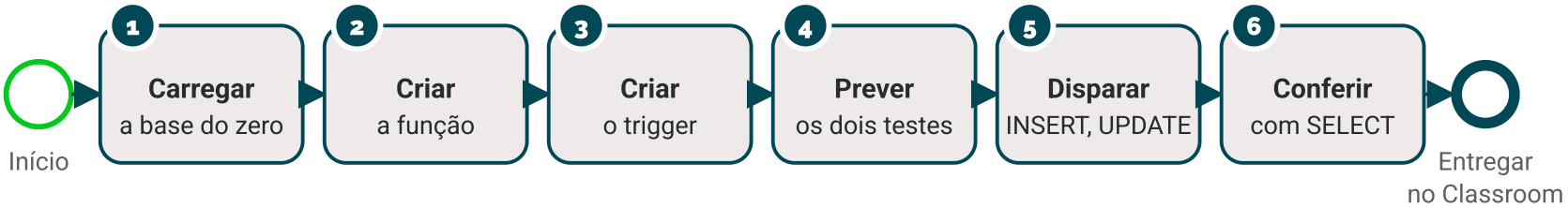
BEFORE

Recusa ou corrige

Roda antes de gravar: RAISE EXCEPTION recusa; NEW.coluna := corrige. Termina com RETURN NEW.

Exercício 10: triggers

Pode fazer em dupla. Cada aluno entrega o seu, no Classroom



> Os triggers 1 a 3 são AFTER; os 4 a 7, BEFORE. O segundo teste não dispara (AFTER) ou é recusado (BEFORE).

Não há prova 3

O que muda no fim do curso

Próxima aula

Prática

Funções, procedures e triggers, com o professor. Sem entrega e sem nota.

Notas

Três provas

As provas 1, 2 e 4, com o mesmo peso.

Prova 4

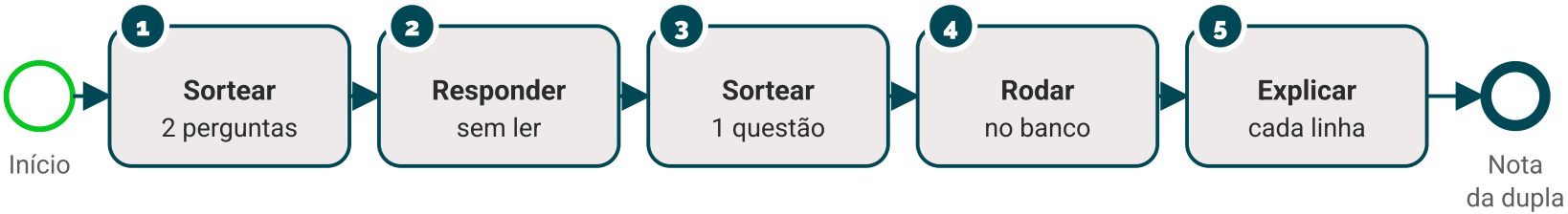
Na mesa do professor

Em dupla: 2 perguntas de teoria e 1 questão dos exercícios 8, 9 e 10.

As perguntas de teoria saem de um banco de perguntas que a turma recebe antes da prova.

A prova 4, passo a passo

A dupla vai à mesa do professor



> Cada pergunta vale 2 pontos e a questão vale 6. Prepare com tudo o que quiser; na mesa, responda com as suas palavras.

O que ainda falta treinar?

Em dupla, nos exercícios 8, 9 e 10

Exercício 8

Funções

Marque: sei fazer, faço com ajuda ou ainda não sei.

Exercício 9

Procedures

Marque: sei fazer, faço com ajuda ou ainda não sei.

Exercício 10

Triggers

Marque: sei fazer, faço com ajuda ou ainda não sei.

O que ficou «ainda não sei» é o que a dupla treina na próxima aula.

Para fixar: responda de memória

Sem olhar o material e sem nota. Escreva as respostas em comentários, no fim do arquivo

Pergunta 1

OLD e NEW

O que OLD e NEW guardam?

Pergunta 2

BEFORE ou AFTER

Qual pode recusar o dado? Por quê?

Pergunta 3

Quem chama

Quem chama o trigger?